

Don't worry at all—building this prototype does **not** require training complex deep learning models from scratch.

The core guideline for Round 2 is: The LLM should NOT be the source of quantitative truth. This means your system will rely on simple, deterministic code (Python/SQL/rules) for the math, and use an LLM (via an API like OpenAI, Gemini, or Claude) solely for contextual reasoning and narrative synthesis.

The Master Blueprint: How to Build It Without Complex ML

[Sample Data: CSVs / SQLite]

|



[Stage 1: Deterministic Engine (Python)] → Calculates KPIs, flags anomalies, breaks down drivers

|



[Stage 2: Synthesis Engine (LLM API)] → Reads findings + text notes, drafts persona stories

|



[Stage 3: Interactive UI (Streamlit)] → Demo app showcasing the 4 mandatory scenarios & telemetry

5-Phase Step-by-Step Roadmap

Phase 1: Create Your Simulated Business Dataset & Semantic Contract

Goal: Satisfy the requirement for 3–5 connected KPIs across 2–3 data sources.

1. **Pick an intuitive domain:** An E-commerce / Retail brand (e.g., selling apparel/electronics).
2. **Define your 4 connected KPIs:**
 - **Revenue** = Traffic × Conversion Rate × Average Order Value (AOV)
 - **Gross Margin** = Revenue — Cost of Goods Sold (COGS) — Shipping Costs
 - **Customer Acquisition Cost (CAC)** = Ad Spend ÷ New Customers
 - **Customer Satisfaction (CSAT)** = Survey Score (1 to 5)

3. Create 3 small CSV files (or SQLite tables) with different grains/refreshes:
 - orders_daily.csv: Daily sales, revenue, units, discount %, return status.
 - marketing_campaigns_weekly.csv: Weekly spend, clicks, impressions by channel.
 - unstructured_logs.csv: Text logs containing customer reviews, regional weather alerts, competitor pricing alerts, and website incident reports.
4. Draft the Semantic Contract (a simple JSON or Markdown table):
 - Formal definition of each metric, formula, owner, update frequency, and who has permission to see it (e.g., Executive vs. Store Manager).

Phase 2: Build the Deterministic "Math & Anomaly" Layer (Pure Python)

Goal: Handle noise detection and driver decomposition with standard logic, not LLM guesses.

1. **Anomaly & Noise Detection:**
 - Calculate a 30-day rolling average and standard deviation (Z-score) for your KPIs.
 - If $\text{Metric} < \text{Mean} - (2 \times \text{StdDev})$ and business impact is $> \$10,000$, flag it as a **material anomaly**.
2. **Mathematical Driver Analysis:**
 - Use contribution math to pinpoint what caused a revenue drop:
$$\Delta \text{Revenue} = \Delta \text{Traffic} + \Delta \text{Conversion} + \Delta \text{AOV}$$
 - Output a clean JSON summary (e.g., {"metric": "Revenue", "drop": "-8.2%", "primary_driver": "Conversion Rate (-12%)", "sub_driver": "Northeast Region"}).

Phase 3: Build the Contextual Synthesis & Narrative Engine (LLM API)

Goal: Combine mathematical output with unstructured text and enforce guardrails.

1. **Context Retrieval (RAG/Lookup):**
 - When an anomaly is detected in the Northeast region during Week 4, filter unstructured_logs.csv for tags matching "Northeast" and "Week 4".

2. Prompt Engineering for Persona-Specific Stories:

- **Persona A (VP of Sales):** Concise, 3-sentence executive summary focusing on macro impact, revenue loss, and strategic actions.
- **Persona B (Regional Operations Lead):** Detailed technical breakdown with warehouse dispatch errors, logistics notes, and ground-level actions.

3. Enforce the Prescriptive Action Framework:

- Structure recommendations as:

Driver → Controllable Lever → Action → Expected Impact → Owner → Confidence
→ Monitoring Plan[cite: 5]

4. Build the Confidence & Abstention Logic:

- If supporting logs are missing or contradictory, instruct the prompt to return: "Confidence: Low (32%). Abstaining from definitive recommendation. Please clarify: Was there an untracked regional campaign?"

Phase 4: Build the Interactive Demo UI (Streamlit)

Goal: Deliver a functional working prototype with telemetry.

Streamlit is a lightweight Python framework where you can build an interactive web app in just a few lines of code. Your app should have tabs for the 4 required test scenarios:

Tab / Scenario	What It Demonstrates in the UI PDF
Scenario 1: Multi-Factor Drop	Shows an 8% revenue drop caused by a checkout glitch (structured) + competitor flash sale (unstructured).
Scenario 2: Low-Confidence / Abstain	Shows a sudden metric spike with zero supporting logs; AI explicitly abstains and prompts for human input.
Scenario 3: Sparse History (New Product)	Demonstrates how the system handles a product launched only 5 days ago (uses cohort benchmarks rather than historical baseline).

Tab / Scenario	What It Demonstrates in the UI PDF
Scenario 4: Role-Based Security	Toggle between "Executive" and "Store Analyst" to see different narrative depth and restricted data views.
Live Telemetry Box	Displays execution latency (e.g., 420ms), model calls, token count (312 tokens), and estimated query cost (\$0.0006).

Phase 5: Final Deliverables Preparation

Goal: Complete all three required Round 2 competition components.

1. **Detailed Business Proposal Document:** Cover target personas, ROI calculations, a 3-phase rollout roadmap, and risk mitigations.
2. **Working Prototype:** The live Streamlit app running locally or deployed on Streamlit Community Cloud.
3. **Pitch Deck & Presentation:** Slides walking through the architecture, business value, and a recorded/live screen demo of your 4 scenarios.

Here is a comprehensive master blueprint for **Track 3: BusinessIntelligence.ai** for Round 2. It breaks down every single requirement, complexity, and expectation from the brief into a practical, implementable technical solution.

1. Recommended Tech Stack

To keep the system modular, audit-proof, and fast to develop (without needing complex deep-learning infrastructure), use a hybrid Python stack:

[Layer 1: Data & Storage] → DuckDB / SQLite + Pandas (Local, fast SQL engine for structured tables + CSVs)

[Layer 2: Deterministic Math] → SciPy / Statsmodels + Custom Python Rules (Z-Score anomaly detection, Contribution Trees)

[Layer 3: Semantic & Security] → JSON Semantic Contract + Role-Based Access Control (RBAC) filtering

[Layer 4: AI & Synthesis] → LLM API (OpenAI GPT-4o-mini, Anthropic Claude 3.5 Sonnet, or Gemini 1.5 Flash) via LangChain / LiteLLM

[Layer 5: Frontend & Telemetry] → Streamlit (Interactive UI with live token, latency, and cost telemetry)

2. Tackling the 8 Round 2 Objectives

Round 2 Objective PDF	Our Corresponding Technical Solution
1. Detect & prioritize material movements	Two-Tier Filter: Use dynamic rolling Z-scores ($Z > 2.0$) for statistical significance combined with an absolute threshold (e.g., $ \Delta\text{Impact} > \$10,000$) to eliminate immaterial noise.
2. Reconcile heterogeneous sources	Unified Data Pipeline: DuckDB joins daily transactional sales tables (daily grain) with weekly marketing spend (weekly grain) and tags unstructured incident/feedback logs using standardized time windows and entity keys (Region, SKU).
3. Identify & rank explanatory drivers	Deterministic Contribution Tree: Calculate exact percentage contributions mathematically ($\Delta\text{Metric} = \sum \Delta\text{Sub-components}$). Drivers are ranked $1 \dots N$ by absolute variance contribution before reaching the LLM.
4. Persona-specific narratives with evidence	System Prompt Templates + Citation Tags: Prompt filters output depth by role (e.g., Executive vs. Store Manager) and enforces footnote citations linking directly to source tables and log IDs.
5. Communicate uncertainty & abstain	Confidence Gate: If supporting data is missing, conflicting, or statistical confidence $< 60\%$, the system sets status to ABSTAIN, generates 2–3 ranked hypotheses, and prompts human review.
6. Practical action recommendations	Structured Playbook Mapping: Maps flagged root causes to business levers in the mandatory format: Driver → Lever → Action → Impact → Owner → Confidence → Monitoring[cite: 6].

Round 2 Objective PDF	Our Corresponding Technical Solution
7. Learn from user feedback	Feedback Capture Loop: Streamlit UI includes 👍 / 🗣️ buttons and correction text boxes that log feedback into a feedback_store.json to tune future prompt context and anomaly sensitivity.
8. Security, cost, latency constraints	Lean Architecture: Pre-computes math in Python (fast, free) and only calls lightweight LLMs for final text formatting, keeping latency < 2s and cost < \$0.002 per insight.

3. Tackling Real-World Complexities

- Multi-Interacting Drivers (Price, Volume, Mix, Supply, Marketing):
 - *Our Solution:* Use a mathematical decomposition formula ($\text{Revenue} = \text{Price} \times \text{Volume} \times \text{Product Mix}$). The script computes the exact delta for each driver independently so multiple overlapping causes don't get confused.
- Varying Refresh Cadences & Grains:
 - *Our Solution:* Maintain metadata tags for data freshness (e.g., orders_table: real-time/daily, marketing_table: weekly-batch-updated). The engine flags when an analysis relies on asynchronous or lagging data.
- Inconsistent KPI Definitions & Semantic Hierarchies:
 - *Our Solution:* A centralized semantic_contract.json serves as the single source of truth for metric names, formulas, aggregation rules, and calendar definitions (e.g., Fiscal vs. Calendar weeks).
- Sparse History for New Products/Categories:
 - *Our Solution:* When time-series history is < 14 days, the engine flags SPARSE_HISTORY mode and swaps time-series anomaly detection for **Cohort/Category Benchmarking** (comparing the new SKU against category averages).
- Contradictory Evidence & Confidence Calibration:
 - *Our Solution:* If structured metrics show a drop due to stockouts, but unstructured logs claim low customer demand, the engine calculates a weighted confidence score, displays both conflicting signals, and flags the ambiguity.

- **Role-Based Security & Data Privacy (RBAC):**
 - *Our Solution:* Implement user role filtering at the data retrieval layer before passing context to the LLM, preventing unauthorized access to restricted columns (e.g., employee profit margins or PII).
- **LLM Economics (Cost, Tokens, Latency, Caching):**
 - *Our Solution:* Cache deterministic anomaly results. When LLM calls are necessary, pass structured, pre-filtered JSON payloads rather than raw data dumps to minimize token consumption.

4. Minimum Prototype Expectations (The 10 Deliverables Checklist)

Here is how each prototype expectation from the brief is implemented in code and UI:

#	Prototype Requirement PDF	Concrete Implementation in Prototype
1	3–5 Connected KPIs across 2–3 sources	Model Revenue, Gross Margin, CAC, and CSAT across orders_daily.csv, marketing_weekly.csv, and unstructured_logs.csv.
2	Lightweight KPI / Semantic Contract	A dedicated JSON/YAML schema explicitly declaring calculation formulas, driver trees, thresholds, data lineage, and access tiers.
3	At least 2 User Personas	• VP of E-Commerce: Strategic summary, EBITDA impact, cross-departmental levers. • Regional Operations Manager: Tactical unit counts, warehouse delays, shipping carriers.
4	Multi-Factor KPI Movement Scenario	Scenario where an 8.2% revenue drop is decomposed into: Checkout payment failure (-5.1%) + Competitor flash sale (-3.1%).

#	Prototype Requirement PDF	Concrete Implementation in Prototype
5	Low-Confidence / Abstention Scenario	An anomalous traffic spike occurs with zero matching logs or campaign data; system explicitly outputs Confidence: 34% (ABSTAIN) and requests human clarification.
6	Sparse-History Scenario	SKU launched 4 days ago; system detects sales below peer launch baselines rather than historical averages.
7	Role-Based Security Scenario	Toggle user role in Streamlit UI: "Analyst" sees regional margin details; "Store Associate" has margin columns redacted.
8	Evidence & Lineage Traceability	Every generated insight includes an expandable "Evidence Drawer" showing source timestamp, analytical formula used, and raw log excerpts.
9	Clear LLM vs. Non-LLM Breakdown	UI badge and technical architecture diagram clearly highlighting deterministic math vs. generative text synthesis.
10	Runtime Telemetry Dashboard	Live metric cards in the UI displaying: Latency: 480ms, Tokens: 382, Model: GPT-4o-mini, Est. Cost: \$0.00057.

5. Next Steps

With this complete mapping locked in, we can proceed step-by-step:

1. **Step 1:** Create the synthetic dataset schema (the 3 CSVs) and the semantic_contract.json.
2. **Step 2:** Write the deterministic Python anomaly and contribution detection functions.
3. **Step 3:** Design the structured prompts and the abstention/guardrail logic for the LLM.

4. **Step 4:** Assemble the Streamlit dashboard to demonstrate all 4 required scenarios and runtime telemetry.
5. **Step 5:** Draft the Round 2 Business Proposal document and pitch slides.

I completely understand the need to zoom in. It is totally normal to want a granular, step-by-step list when tackling a project like this for the first time!

To clarify your question about the data: we are *not* scraping real data from a website. The competition guidelines explicitly state that you "are not expected to have access to a real company's proprietary data" and that using "illustrative or sample data is expected and encouraged". We will generate simulated CSV files ourselves to keep things simple and fully under our control.

Here is the micro-task breakdown of our roadmap:

1. Data Setup & Semantic Contract

- Write down the exact mathematical formulas for your chosen 3 to 5 connected KPIs.
- Generate three simulated CSV files (e.g., daily orders, weekly marketing, unstructured logs) to represent two or three data sources with different refresh cadences.
- Draft a semantic_contract.json file that defines calculations, thresholds, lineage, and access restrictions.

2. The Deterministic Math Engine (Python)

- Write a Python function calculating 30-day rolling averages and Z-scores for baseline anomaly detection.
- Code a deterministic contribution analysis script that calculates exactly how much a specific sub-driver impacted the top-line metric.
- Format the mathematical outputs of these functions into a clean, structured JSON payload to feed to the AI.

3. LLM Synthesis & Guardrails

- Set up a basic LLM API connection (e.g., OpenAI or Gemini) using standard Python libraries.
- Design specific prompt templates that adjust the depth of the insight narratives for at least two different user personas.

- Code the abstention logic so the AI explicitly requests clarification or abstains when evidence is contradictory or insufficient.

4. Interactive UI & Telemetry (Streamlit)

- Initialize a basic Streamlit web app with tabs for the mandatory test cases, such as the multi-factor KPI movement and the sparse-history (newly launched KPI) scenario.
- Embed a live telemetry dashboard in the UI to track latency, model calls, token usage, and estimated cost.
- Record a walkthrough of this functional prototype demonstrating all required scenarios for your pitch presentation.